

WEEK 3 — Understanding & Conceptualizing Interaction (Full explanation)

Big picture: what Week 3 is trying to teach you

Week 3 is saying:

Before you design screens and buttons, you must first **understand the problem space**, challenge your **assumptions and claims**, and then build a **conceptual model** of the system: what users will do, what objects exist, how actions work, and what interaction type fits best.

And to do that properly, you use structured tools:

- **QOC** (Question–Option–Comment): design rationale, trade-offs
 - **HTA** (Hierarchical Task Analysis): break user goals into steps
 - **STN** (State Transition Network): represent system states + transitions
- All three are included in your week's content.
-

PART A — Task Analysis Techniques

1) QOC — Question, Option, Comment (Design rationale)

What QOC is

QOC is a semi-formal way to explore the design space and record **why you chose something** (your rationale).

It makes decision-making explicit, which is good for teamwork and evaluation.

Structure (MEMORIZE)

- **Question:** the design problem (what decision must be made?)
- **Option:** alternative solutions (possible answers)
- **Comment:** pros/cons or criteria/argument explaining trade-offs

Think of QOC like: “We’re deciding X. Here are options A/B/C. Here’s what we gain/lose with each.”

Examples (from your slides/notes)

Example: Login method

- Question: How should users log in?
- Options: password vs biometrics vs SSO
- Comments: password familiar but forgettable/hackable; biometrics convenient but privacy/hardware; SSO low friction but depends on third-party.

Example: Navigation in a mobile app

- Bottom bar: visible/easy but limited items
- Hamburger: hides options (less clutter) but hurts discoverability
- Gestures: natural but learnability/accessibility issues

Why QOC is important

It prevents “random design decisions.” It forces you to justify choices based on user needs and trade-offs, not personal preference.

What to MEMORIZE (QOC)

- Definition of QOC + why it’s used (design rationale, exploring design space)
 - Exact structure: Question / Option / Comment (criteria)
 - Be able to apply it to a simple design decision (login/navigation/errors).
-

2) HTA — Hierarchical Task Analysis

What HTA is

HTA is a structured way to describe **how work is performed**, by breaking a goal into sub-goals and steps (and then steps into smaller steps).

Key components (MEMORIZE)

- **Goal:** what user wants to achieve
- **Sub-tasks:** steps needed to achieve it
- **Plan:** the order/conditions (e.g., “do 1.1 then 1.2 then 1.3, then do 2”)

Why HTA matters

HTA helps you:

- see complexity of tasks
- locate where users might fail or get confused
- design UI flows that match real workflows

Example: Online shopping checkout (from notes)

Goal: “Buy a product online” broken into search → add to cart → checkout → confirmation, with smaller subtasks like typing keywords, browsing results, entering shipping, selecting payment.

What to MEMORIZE (HTA)

- Definition: hierarchy of tasks to achieve a goal
 - Goal / subtask / plan
 - Why plans matter (sequence + conditions, not just a list).
-

3) STN — State Transition Network (Finite State Machine)

What STN is

An STN models how a system moves between **states** in response to **actions/events**. It’s basically the “behavior diagram” of a system.

Components (MEMORIZE)

- **States**: conditions of the system (e.g., logged out, logged in)
- **Transitions**: arrows showing change from one state to another
- **Actions/events**: triggers that cause transitions (button click, sensor event)

Examples in your notes

Bottle in bottling plant: states like empty/full/broken/sealed; actions like squirt, cap, break.

Chess game: start → white turn → black turn → win/lose/draw based on actions like moves/checkmate/stalemate.

Why STN matters in HCI

Because interfaces are “dialogues” between user and system. STN ensures you don’t forget states like:

- What if the user cancels?

- What if login fails?
- What if the internet drops?
That's exactly where usability bugs come from.

What to MEMORIZE (STN)

- STN = finite state machine model for system behavior
 - State / transition / action-event.
-

PART B — Conceptualizing Interaction (Problem space → Design space)

4) Conceptualizing design: Why we do it (Robot Waiter story)

The robot waiter example (two versions)

The lecture gives a design example:

Idea 1: A robot waiter that takes orders and entertains customers by conversation.

This sounds cool, but it's full of unknowns:

- How will it move to look like it's talking?
- Will customers find it pleasant?
- Will it become annoying or a fad?

Idea 2: A functional robot server (tool-like) that delivers trays from kitchen to preset area; customers pick items; saves waiter time.

This addresses a real-world problem and is less risky.

Why conceptualize?

Because early design ideas are vague, and conceptualizing forces you to check:

- feasibility ("can we realistically build this?")
- desirability ("do people want this?")
- usefulness ("does it solve a real problem?")

Assumption vs Claim (VERY testable)

What is an assumption? (MEMORIZE)

Assumption = taking something for granted that still needs investigation.

Example: “People will enjoy the robot having a conversation.”

What is a claim? (MEMORIZE)

Claim = stating something is true even though it’s still open to question.

Example: “It will raise revenue by attracting more people.”

This is important because many “cool ideas” die when assumptions and claims fail in reality.

Working through assumptions (how designers should behave)

Early stage questions: where did idea come from? what research/theory supports it? ask questions, reconsider assumptions, articulate concerns.

Poor assumptions are hard to articulate; that’s a red flag: the idea may be vague/unrealistic.

Framework for analyzing problem space (MEMORIZE list)

Your slides literally list the “problem space question framework”:

- Are there problems with an existing product/UX? what are they?
- Why do you think there are problems?
- How might your design ideas overcome these?
- If new UX, how does it support/change/extend current practices?

3D TV activity (assumptions/claims)

The lecture shows how 3D TV was a “new experience” rather than solving an existing problem; assumption: people enjoy enhanced detail; claim: people will pay more for 3D experience.

Benefits of conceptualizing (MEMORIZE)

- **Orientation:** ask specific questions about conceptual model understanding
- **Open-minded:** avoid narrow focus too early
- **Common ground:** agreed terms among the team

5) From problem space to design space

Once you understand the problem space, you can make better design decisions: interface, behavior, functionality. But before deciding, you need a **conceptual model**.

6) Conceptual Model (core Week 3 topic)

Definition (MEMORIZE EXACTLY)

A conceptual model is a **high-level description of how a system is organized and operates**. It helps designers “straighten out thinking before laying out widgets.”

So: conceptual model is NOT the UI screens. It’s the underlying logic: objects, actions, relationships.

Components of a conceptual model (MEMORIZE)

- **Metaphors and analogies** (help users understand what it is and how to use it)
- **Concepts** exposed through product: objects, attributes, operations (e.g., save, organize, revisit)
- **Relationships/mappings** between concepts

First steps when creating a conceptual model (MEMORIZE as questions)

- What will people do to carry out tasks?
- How will the model support these tasks?
- What interface metaphor fits (if any)?
- What interaction modes/styles?
- How will users understand the conceptual model?

Best conceptual models

The best ones feel “obvious and simple” and operations feel intuitive.

7) Interface metaphors (part of conceptual modeling)

What an interface metaphor is (MEMORIZE)

An interface metaphor makes the interface similar to a physical entity/activity to exploit familiar knowledge and help users understand unfamiliar functionality.

Examples:

- “Surfing the web” (activity metaphor)
- Desktop metaphor (files/folders/trash)
- Shopping cart icon for selecting items

The card metaphor (popular)

Cards are popular because they are familiar, sortable, “flickable,” chunk content meaningfully (“card sized”) and help organization.

Benefits of interface metaphors (MEMORIZE)

- easier learning
- familiar action helps understand conceptual model
- can enable innovation and accessibility for diverse people

Problems with interface metaphors (MEMORIZE list)

- may break cultural/conventional rules (recycle bin on desktop)
 - can constrain designers’ thinking and imagination
 - can transfer bad designs (copy bad parts)
 - often don’t scale well for complex systems
-

8) Interaction Types (what user is “doing”)

The five main interaction types (MEMORIZE list)

1. **Instructing**: giving commands/selecting options
2. **Conversing**: interacting like a conversation (voice/NLP/chatbots)
3. **Manipulating**: direct manipulation of objects (drag/zoom/open/close)
4. **Exploring**: moving through virtual/physical environments (VR/AR, maps)
5. **Responding**: system initiates (notifications, proactive alerts)

Deep dive: Direct Manipulation (Shneiderman)

Direct manipulation has 3 core properties (MEMORIZE):

- continuous representation of objects/actions
- physical actions (click/drag) instead of complex commands
- rapid reversible actions + immediate feedback

Benefits: novices learn quickly, experts fast, fewer error messages, users feel control/confidence.

Disadvantages: metaphor taken literally; not all tasks suit objects; can waste screen space; cursor movement sometimes slower than keyboard shortcuts; some tasks better delegated (spellcheck).

Choosing an interaction type (MEMORIZE)

- Direct manipulation: good for “doing” tasks (drawing, flying, driving)
- Instructing: repetitive tasks (file mgmt, spellchecking)
- Conversing: requesting info, music/TV, learning
- Hybrid models: multiple ways to do same action

Interaction type vs Interface style (VERY testable)

- **Interaction type** = what user is doing conceptually (instructing, conversing, exploring...)
- **Interface style** = what interface supports it (command-line, menu-based, gesture, voice...)

This difference is frequently asked.

9) Other forms of conceptual knowledge (for design + research framing)

Your lecture says conceptual knowledge that informs design includes: paradigms, challenges, visions, theories, models, frameworks.

Paradigms

A general approach adopted by a community; shared assumptions, concepts, values, practices. Examples: desktop, ubiquitous computing, IoT, human-centered AI.

Challenges

Societal challenges like SDGs (zero hunger, education, gender equality) and climate change; HCI addresses tech values that affect society’s behavior (disposal/recycling).

Visions

Future-looking scenarios framing R&D (knowledge navigator, smart cities, smart health, human-centered AI) and raising ethics like privacy/trust.

Theory (MEMORIZE)

Theory explains a phenomenon; helps identify relevant factors (cognitive/social/affective); predicts what users do with interfaces.

Roles of theory in HCI (MEMORIZE list)

Descriptive, explanatory, predictive, prescriptive, generative, informative, conceptual, critical.

Models

Simplification to predict/evaluate designs (e.g., Norman's Seven Stages of Action).

Frameworks

Interrelated concepts/questions advising what to look for and how to design experiences; can be steps/questions/principles/tactics/dimensions.

10) Classic framework: Norman's 3-model relationship (Designer / System Image / User)

This is on your summary pages and is a classic exam target.

The three components (MEMORIZE)

1. **Designer's Model:** how the designer thinks the system should work
2. **System Image:** how the system is presented to user through interface, manuals, help etc. (this is the only thing user can actually see)
3. **User's Model:** the user's understanding formed through interacting with the system image

Goal

Make the System Image accurately reflect the Designer's Model so the User's Model matches it.

Week 3 Summary (in 6 lines)

Week 3 teaches that good design starts by understanding the problem space, testing assumptions and claims, and building a conceptual model. Conceptual models define what users can do, what concepts they need, and how concepts relate. Interface metaphors can help learning but can also constrain design. Interaction types describe what the user is doing (instructing, conversing, manipulating, exploring, responding), and you must distinguish them from interface styles. QOC captures design rationale, HTA breaks tasks into steps and plans, STNs model system behavior. Paradigms/visions/theories/models/frameworks help frame research and design.

WHAT TO MEMORIZE (Week 3 “Ratta List”)

If your teacher nitpicks, these are the exact bullets that usually appear:

1. QOC definition + structure: Question/Option/Comment + “trade-offs/rationale”
 2. HTA definition + Goal/Subtasks/Plan
 3. STN definition + State/Transition/Event
 4. Difference: **assumption vs claim** (wording matters)
 5. Problem space framework questions (4 questions list)
 6. Conceptual model definition (high-level description of organization/operation)
 7. Components of conceptual model: metaphors, concepts (objects/attributes/operations), relationships
 8. Interface metaphor definition + benefits + problems list
 9. Five interaction types list + direct manipulation 3 properties
 10. Interaction type vs interface style (difference)
 11. Paradigm vs vision vs theory vs model vs framework (what each is)
 12. Norman framework: Designer's model / System image / User's model + goal
-

Practice Questions (with answers + explanation)

A) 5 MCQs

MCQ 1

In QOC, the “Comment” mainly represents:

- A) The UI layout
- B) The trade-offs/arguments for and against options
- C) The system’s states
- D) The user’s persona

Answer: B

Why: Comment is the rationale (pros/cons/criteria) that justifies options.

MCQ 2

Which method is best for breaking a user goal into sub-goals and specifying a “plan” order?

- A) QOC
- B) HTA
- C) STN
- D) Interface metaphor

Answer: B

Why: HTA includes goal, subtasks, and plan (sequence/conditions).

MCQ 3

A statement like “This robot waiter will increase restaurant revenue” is a:

- A) Assumption
- B) Claim
- C) Paradigm
- D) Interface style

Answer: B

Why: It asserts truth that is still open to question → claim.

MCQ 4

The three core properties of Direct Manipulation (Shneiderman) include:

- A) Menu-based commands, no feedback, no undo
- B) Continuous representation, physical actions, rapid reversible actions with feedback
- C) Only voice interaction and predictions
- D) Only keyboard shortcuts

Answer: B

Why: That’s the exact trio in the lecture.

MCQ 5

“Interaction type” differs from “interface style” because:

- A) Interaction type is hardware, interface style is software
- B) Interaction type describes what user is doing; interface style is the interface used
- C) Interaction type is always menus; interface style is always voice
- D) They are the same thing

Answer: B

Why: Interaction type = concept of activity; interface style = command/menu/gesture/voice etc.

B) 5 Short Answer Questions (with answers)

SAQ 1: Define assumption and claim with one example each.

Answer:

- Assumption: something taken for granted that needs investigation (e.g., “customers will enjoy talking to a robot”).
- Claim: statement asserted true but still questionable (e.g., “robot will raise revenue”).

Explanation: Teachers check if you understand “assumption = taken for granted” vs “claim = asserted true.”

SAQ 2: What is a conceptual model and why do designers make one before UI design?

Answer: A conceptual model is a high-level description of how a system is organized and operates. Designers use it to clarify thinking and define concepts/relationships before making widgets/screens.

Explanation: It prevents building a UI with no clear underlying logic.

SAQ 3: List the components of a conceptual model.

Answer: Metaphors/analogies; concepts (domain objects, attributes, operations); relationships/mappings between concepts.

Explanation: This is a direct memorization question.

SAQ 4: What are the five interaction types? Give one example for any two.

Answer: Instructing, Conversing, Manipulating, Exploring, Responding.

Examples: Manipulating = dragging/zooming objects; Responding = fitness tracker alerts.

Explanation: You must know the list; examples show understanding.

SAQ 5: Explain Norman’s Designer’s Model, System Image, and User’s Model in 2–3 lines.

Answer: Designer’s model is how designer intends system to work. System image is what user sees (interface/docs/help). User’s model is what user believes about system formed from system image. Goal is system image should reflect designer’s model so user’s model matches.

C) 1 Case Study (with answer)

Case Study: “Login + Navigation redesign” using QOC + conceptual model

You’re designing a campus app with: attendance, grades, fee payment, course announcements. Students complain: “I can’t find things fast” and “login is annoying.”

Task (what the teacher may ask):

1. Create a **QOC** for login.
2. Suggest an interaction type for navigation and justify.
3. Identify one risky **assumption** and one **claim** in your design proposal.
4. Explain what should be shown in the **System Image** to support the User’s Model.

Model Answer

1) QOC for login (simplified)

- **Question:** How should users log in?
- **Options:**
 - Username+Password
 - Biometrics
 - Single Sign-On (Google)
- **Comments/trade-offs:**
 - Password: familiar but forgettable + security risk
 - Biometrics: convenient but privacy + needs hardware
 - SSO: low friction but dependency on third party

2) Interaction type choice

For navigation in an app, the main user activity is usually **instructing** (selecting options) plus some **manipulating** (swiping cards, dragging). A bottom navigation bar supports instructing well because it makes options visible and quick.

3) One assumption + one claim

- Assumption: “Students will prefer biometric login and trust it.” (needs investigation)
- Claim: “This redesign will reduce time-to-find-features by 50%.” (must be tested)

4) System Image support (Norman)

To help user form correct mental model, the System Image should clearly show:

- what each tab does (labels/icons),
- feedback on successful login,
- where to find key features (“Grades” visible; not hidden),

- short onboarding tooltip (“Fees are in Payments tab”).
This makes user’s model match designer’s intended model.